

Aegis

A verifier you can trust for AI security patches

A deterministic verifier that judges whether a security patch *genuinely* closes a vulnerability — and measures its own reliability, including where it fails.

Cody Lee · June 2026 · github.com/clee12111/Aegis

The problem

Security-agent benchmarks grade a patch by a shallow proxy: did the test suite pass, did the one known exploit stop firing. Nobody checks whether the fix actually *closed* the vulnerability or merely satisfied the single test it was shown. A March 2026 study (PVBench) found that roughly 40% of patches accepted by these checks are still broken. The weak point is the measurement layer, not the model. Aegis builds that layer: a deterministic verifier that judges whether a patch is real, and — the part the field omits — quantifies its own reliability.

Two layers

The system is two measurement layers stacked. **Layer 1** judges a patch: genuine or gamed. **Layer 2** judges the verifier itself — its precision and recall on a labeled gold set, i.e. how often the verifier is right. A verifier without Layer 2 is only an assertion (“trust my verdicts”); with it, it is a measured instrument with a stated error rate and stated blind spots. That reliability number is what no public security benchmark reports.

How a verdict is reached

The core is vulnerability-agnostic; a vulnerability enters only as a plugin that supplies four things: a behavioral oracle, an exploit family, candidate patches, and a class taxonomy. Two signals decide the verdict. First, the patch must block a **held-out** exploit family it never saw — a disjoint labeling/verifier split plus a grammar-and-mutation fuzzer catch patches that merely memorized the one known exploit. Second, it must keep legitimate inputs working — which catches the “fix” that simply deletes the feature. The oracle is deterministic and behavioral: it checks real behavior (did the bytes escape the directory? did an injected command actually run?), not whether a test went green.

It knows when it cannot judge

The verifier does not always answer. Before blessing a patch as genuine, it checks whether its own attack suite can even establish the vulnerability and spans the known attack classes. If it cannot, it returns **ABSTAIN** — “I have not tested this, so I will not certify it” — gated on an objective coverage check, never a tunable confidence knob (a knob would let the verifier dodge hard cases to flatter its own accuracy). Crucially this is *honest deferral*, not error-hiding: in testing, every patch the verifier deferred resolved to the correct verdict once the missing coverage was added.

The findings that land

The verifier caught two of the most widely-recommended security fixes as only *partially* correct — each defends the obvious shape of an attack and silently leaks a sibling shape of the same class:

[normpath](#) (path traversal). The canonical “normalize the path and reject leading `../`” fix passed every string-based test for the life of the project — until the verifier tried a symlink whose filename was perfectly innocent. [normpath](#) is string manipulation; it cannot see that a directory in the path is a symlink pointing

outside the allowed folder. It fell.

`shlex.quote` (command injection). The fix every developer is taught for shell injection. It escapes shell metacharacters, so it blocks command injection — but it passes an input such as `--output=/etc/cron.d/evil` straight through as a command-line *flag*. Quoting a flag does not stop it being a flag. The verifier caught it; so did `subprocess` list-form, for the same reason.

Validated on real CVEs

The verifier then left synthetic gold sets and judged real, disclosed CVEs on live Dockerized targets:

Target / CVE	The gamed patch	Test suite	Verifier
MLflow CVE-2024-1558	URL-decode then reject “..” — a plausible fix a reviewer would accept	PASSES (148 / 148)	GAMED
LibreChat CVE-2024-11170	A real, maintainer-shipped fix (the actual patch for a sibling CVE)	PASSES	GAMED
(both) the official fix	—	passes	GENUINE

The MLflow case shows the headline plainly: a fix that passes the entire functional suite is still exploitable (via the `%23` fragment trick), and test-suite verification accepts it while the verifier rejects it. The LibreChat case is the strongest form of the result: the gamed patch is a genuine security improvement — it correctly closes a sibling CVE and passes the tests — yet leaves the same traversal *class* open in the upload handler, and a second, separately-disclosed CVE is the independent proof it was incomplete. No “we wrote the fake” objection survives.

Both ran with **zero changes to the verifier core**. The same core scored four vulnerability plugins — two synthetic CWE families and two real CVEs — making transferability a validated property of the architecture, not a one-off.

Honest limits — the contribution is knowing them

The verifier is never absolute ground truth; it is a *characterized approximation*, bounded forever by what its oracle can **see** (a bug with no sanitizer is invisible) and what its attack suite can **reach** (a path never exercised is never tested). Complete, sound verification of an unbounded attack space is provably impossible — the reference-free-verification hardness from the RL-with-verifiable-rewards literature. So the claim is never “this verifier is correct,” but “here is its measured precision/recall, and here is exactly where its coverage ends.” That honesty — measured accuracy, a disclosed boundary, and abstention on the rest — is the contribution, precisely because the rest of the field papers the gap over with “the tests passed.”

Stack. Python · Google Cloud (Compute Engine VM, Linux) · Docker / Kali (BountyBench sandbox) · grammar + mutation fuzzing · deterministic behavioral oracles · git + pre-commit measurement hooks · provider-swappable model APIs.

Method. Mirrors a prior project (Meridian): build the measurement layer first, trust the numbers it produces, and do not claim more than the delta — and the limits — show. Active research project; the agent half (a code-graph localization scaffold scored against published BountyBench baselines) is in progress.